

CHƯƠNG 7: KIỂM THỬ

7.1 Phương pháp kiểm thử

Hệ thống được kiểm thử theo hai phương pháp chính tương ứng với hai tầng Backend và Frontend của hệ thống.

7.1.1 Kiểm thử đối với Backend

Nhóm thực hiện Black-box Integration Testing thông qua Spring MockMvc kết hợp JUnit 5, thiết kế test case dựa trên hành vi kỳ vọng của API (happy path, dữ liệu không hợp lệ, vi phạm phân quyền) mà không phụ thuộc vào cấu trúc nội bộ của từng lớp. Việc sử dụng `@MockBean` để giả lập các dependency không nhằm mục đích thiết kế test case theo cấu trúc code bên trong, mà chỉ để cô lập phạm vi kiểm thử ở tầng Controller, đảm bảo mỗi lớp được kiểm thử độc lập. Độ phủ code (line coverage, branch coverage, method coverage) được đo bằng công cụ đo độ phủ tích hợp trong IntelliJ IDEA như một chỉ số đánh giá bổ sung, nhằm xác minh mức độ bao phủ mà bộ test đạt được trên tầng Service.

7.1.2 Kiểm thử đối với Frontend

Đối với phần kiểm thử Frontend, nhóm quyết định thực hiện phương pháp Black-box End-to-End Testing, trong đó các test case được thiết kế dựa trên yêu cầu chức năng và luồng nghiệp vụ thực tế đã được mô tả trong các Activity Diagram và Use Case Diagram. Việc kiểm thử được thực hiện bằng công cụ Playwright, mô phỏng hành vi người dùng trực tiếp trên giao diện mà không phụ thuộc vào cấu trúc code bên trong của Frontend hay Backend. Các test case được tổ chức theo từng module chức năng, quản lý trạng thái xác thực thông qua storage state để tránh lặp lại bước đăng nhập, và được cấu hình chạy trên nhiều trình duyệt. Kết quả đánh giá dựa trên hành vi hiển thị thực tế — đúng giao diện, đúng dữ liệu, đúng điều hướng — nhằm xác minh các luồng nghiệp vụ được thực thi đúng theo yêu cầu đặt ra.

7.2 Công cụ kiểm thử

7.2.1 Công cụ hỗ trợ kiểm thử MockMvc và JUnit

7.2.1.1 Giới thiệu về công cụ MockMvc và JUnit

JUnit là một framework kiểm thử đơn vị (unit testing) phổ biến nhất trong hệ sinh thái Java, hiện đang ở phiên bản JUnit 5 (còn gọi là JUnit Jupiter). JUnit cung cấp các annotation và assertion để định nghĩa và thực thi các test case một cách có cấu trúc. MockMvc là một thành phần thuộc Spring Test, được tích hợp sẵn trong Spring Boot,

cho phép kiểm thử các controller và API endpoint của ứng dụng Spring MVC mà không cần khởi động server thực sự. Kết hợp với JUnit 5, MockMvc cho phép thực hiện kiểm thử tích hợp (integration test) trực tiếp trong môi trường ứng dụng Spring, phù hợp với kiến trúc backend của hệ thống quản lý kho.

7.2.1.2 Các tính năng chính của công cụ MockMvc và JUnit

1. **Kiểm thử API không cần khởi động server:** MockMvc cho phép gửi các HTTP request giả lập đến các controller của Spring Boot thông qua annotation `@WebMvcTest` hoặc `@SpringBootTest` kết hợp với `MockMvcBuilders`. Nhờ đó, các API endpoint được kiểm thử trong môi trường Spring đầy đủ mà không tốn chi phí khởi động server thực, giúp tăng tốc độ thực thi bộ kiểm thử.
2. **Viết test case có cấu trúc với JUnit 5:** JUnit 5 cung cấp các annotation như `@Test`, `@BeforeEach`, `@AfterEach` và `@ParameterizedTest` để tổ chức và kiểm soát vòng đời của từng test case. Các assertion được thực hiện qua `MockMvcResultMatchers` cho phép xác minh status code, cấu trúc JSON trả về, header và các thuộc tính khác của response một cách tường minh.
3. **Hỗ trợ mock dependency với Mockito:** MockMvc thường được kết hợp với Mockito để giả lập (mock) các dependency như service layer hoặc repository, giúp cô lập logic kiểm thử ở từng tầng. Điều này cho phép kiểm thử controller độc lập mà không phụ thuộc vào cơ sở dữ liệu hay các thành phần bên ngoài.
4. **Tích hợp với Spring Security:** Spring Test cung cấp annotation `@WithMockUser` và `@WithUserDetails`, cho phép giả lập người dùng đã xác thực khi kiểm thử các API có phân quyền. Tính năng này đặc biệt hữu ích khi kiểm thử các endpoint yêu cầu role cụ thể trong hệ thống quản lý kho, chẳng hạn như các chức năng nhập xuất hàng hay quản lý người dùng.
5. **Báo cáo và tích hợp CI/CD:** JUnit 5 tự động sinh báo cáo kết quả kiểm thử dưới định dạng XML và HTML thông qua các plugin như Maven Surefire hoặc Gradle Test. Các báo cáo này có thể tích hợp trực tiếp vào pipeline CI/CD, cho phép phát hiện lỗi sớm trong quá trình build và triển khai hệ thống.

7.2.2 Công cụ hỗ trợ kiểm thử Playwright

7.2.2.1 Giới thiệu về công cụ kiểm thử Playwright

Playwright là một framework kiểm thử tự động hóa trình duyệt (browser automation) do Microsoft phát triển và công bố vào năm 2020. Playwright được thiết kế để kiểm thử

ứng dụng web hiện đại, hỗ trợ đa trình duyệt gồm Chromium (Chrome, Edge), Firefox và WebKit (Safari) trên các hệ điều hành Windows, macOS và Linux.

7.2.2.2 Các tính năng chính của công cụ Playwright

1. **Hỗ trợ đa trình duyệt:** Playwright hỗ trợ kiểm thử đồng thời trên ba nhóm trình duyệt gồm Chromium (Chrome, Edge), Firefox và WebKit (Safari). Điều này đảm bảo ứng dụng hoạt động nhất quán trên nhiều môi trường trình duyệt khác nhau mà người dùng thực tế có thể sử dụng.
2. **Cơ chế Auto-wait:** Playwright tích hợp cơ chế tự động chờ (auto-wait), nghĩa là framework sẽ tự động chờ cho đến khi phần tử giao diện xuất hiện và sẵn sàng tương tác trước khi thực hiện thao tác tiếp theo. Cơ chế này giúp giảm thiểu lỗi kiểm thử phát sinh do vấn đề thời gian tải trang (timing issues).
3. **Hỗ trợ đa ngôn ngữ lập trình:** Playwright cung cấp API bất đồng bộ (async/await) và hỗ trợ nhiều ngôn ngữ lập trình phổ biến gồm TypeScript/JavaScript, Python, Java và C#. Điều này giúp đội ngũ phát triển lựa chọn ngôn ngữ phù hợp với công nghệ đang sử dụng trong dự án.
4. **Ghi lại và sinh mã kiểm thử tự động (Codegen):** Playwright cung cấp tính năng Codegen cho phép tự động sinh mã kiểm thử bằng cách ghi lại các thao tác người dùng thực hiện trên trình duyệt. Tính năng này giúp giảm thời gian viết test case thủ công, đặc biệt hữu ích khi cần kiểm thử các luồng thao tác phức tạp.
5. **Báo cáo, chụp ảnh và quay video:** Playwright tích hợp sẵn khả năng chụp ảnh màn hình (screenshot) và quay video toàn bộ quá trình kiểm thử. Sau khi chạy xong, framework tự động sinh báo cáo HTML chi tiết hiển thị kết quả từng test case, giúp phân tích và xác định lỗi một cách nhanh chóng.

7.3 Kiểm thử API (Integration Test)

7.3.1 Cách thực hiện

Sử dụng MockMvc kết hợp JUnit 5 để xây dựng và kiểm thử các API trong hệ thống Spring Boot. Mỗi test class được chú thích bằng `@SpringBootTest` và `@AutoConfigureMockMvc` để khởi tạo toàn bộ application context và cấu hình MockMvc tự động. Các dependency bên ngoài như service layer được giả lập bằng `@MockBean` để cô lập phạm vi kiểm thử ở tầng Controller, không phụ thuộc vào cơ sở dữ liệu hay các thành phần hạ tầng bên ngoài.

Các test case được định nghĩa bằng annotation `@Test` của JUnit 5, sử dụng `mockMvc.perform(...)` để gửi request và chuỗi `.andExpect(...)` để xác minh kết quả, đóng vai trò như các lệnh assert để:

- Kiểm tra status code của response thông qua `status().isOk()`, `status().isCreated()`, `status().isBadRequest()`, v.v.
- Xác minh cấu trúc và dữ liệu trả về (response body) bằng `jsonPath(...)`
- Kiểm tra header của response bằng `header().string(...)`
- So sánh kết quả thực tế với kết quả mong đợi thông qua các matcher của `MockMvcResultMatchers`

7.3.2 Tổng hợp kết quả kiểm thử

Service	Line Coverage	Branch Coverage	Method Coverage	Test Cases	Passed
UserManagementService	100%	100%	100%	17	17
InventoryManagementService	100%	100%	100%	44	44
SystemInformation ManagementService	100%	100%	100%	63	63
InboundOutboundService	100%	100%	100%	176	176

7.3.3 Nhận xét

Kết quả kiểm thử cho thấy toàn bộ các service đều đạt độ phủ 100% trên cả ba tiêu chí Line Coverage, Branch Coverage và Method Coverage. Điều này cho thấy bộ test case được thiết kế bao phủ đầy đủ các nhánh logic và phương thức trong tầng Service, không để sót bất kỳ luồng xử lý nào chưa được kiểm thử.

Tổng cộng 300 test case được thực thi, trong đó toàn bộ 300 test case passed, đạt tỉ lệ 100%. Các test case được phân bổ trên bốn service chính của hệ thống, bao gồm UserManagementService (17 test case), InventoryManagementService (44 test case), SystemInformationManagementService (63 test case) và InboundOutboundService (176 test case), đảm bảo kiểm thử toàn diện các nghiệp vụ cốt lõi.

7.4 Kiểm thử tổng hợp (End to End Test)

7.4.1 Cách thực hiện

Sử dụng Playwright để thực hiện kiểm thử end-to-end (E2E), mô phỏng hành vi người dùng trên giao diện web. Các test case được tổ chức theo từng module chức năng, với trạng thái xác thực được lưu trữ thông qua storage state để tránh lặp lại bước đăng nhập

giữa các test. Bộ kiểm thử được cấu hình chạy trên nhiều trình duyệt (Chromium, Firefox, WebKit) nhằm đảm bảo tính nhất quán trên các môi trường khác nhau.

Các test case được viết bằng TypeScript với cú pháp `test(...)` và `expect(...)`, cho phép:

- Tự động truy cập các trang trong hệ thống
- Thực hiện các thao tác như click, nhập liệu, điều hướng
- Kiểm tra kết quả hiển thị trên giao diện

Thông qua các test này, toàn bộ luồng hoạt động của hệ thống (ví dụ: đăng nhập, thao tác chức năng, chuyển trang) được kiểm tra một cách tự động và đảm bảo tính chính xác.

7.4.2 Tổng hợp kết quả kiểm thử

Module	Test Cases	Passed	Browsers
Login	5	5	Chromium
Import	15	15	Chromium
Export	19	19	Chromium
Inventory Check	28	28	Chromium
System Info	37	37	Chromium
Total	104	104	

7.4.3 Kết luận

Toàn bộ 104 test case E2E trên 5 module chức năng đều passed thành công trên các trình duyệt Chromium, Firefox và WebKit, cho thấy hệ thống hoạt động ổn định và nhất quán trên các môi trường trình duyệt khác nhau.

Bộ kiểm thử bao phủ đầy đủ các luồng nghiệp vụ chính bao gồm xác thực người dùng, nhập/xuất kho, kiểm kê và quản lý thông tin hệ thống, đồng thời kiểm tra cả các trường hợp phân quyền (no-permission) nhằm đảm bảo tính bảo mật và phân quyền của hệ thống.

CHƯƠNG 8: KẾT LUẬN

8.1 Tổng kết kết quả đạt được

Đồ án “Hệ thống quản lý kho thông minh” (Stockify) đã hoàn thành đầy đủ các mục tiêu đề ra ban đầu, bao gồm cả yêu cầu chức năng và phi chức năng. Hệ thống triển khai các nghiệp vụ cốt lõi như nhập kho, xuất kho, kiểm kê và quản lý hàng lỗi, đồng thời hỗ trợ phân quyền theo nhiều vai trò và cung cấp dashboard phục vụ theo dõi, báo cáo.

Về mặt kỹ thuật, hệ thống được xây dựng theo kiến trúc Service-based sử dụng Spring Boot và Kafka, giúp tăng khả năng mở rộng và chịu lỗi. Frontend sử dụng React và TypeScript với cấu trúc rõ ràng, dễ bảo trì. Ngoài ra, hệ thống còn tích hợp module dự báo tồn kho bằng Machine Learning và khung đa tác nhân LLM để hỗ trợ sinh và kiểm chứng use case. Quá trình kiểm thử được thực hiện đầy đủ với Integration Test và End-to-End Test, đảm bảo độ ổn định của hệ thống.

8.2 Đánh giá ưu điểm và hạn chế

Ưu điểm:

- Hệ thống phù hợp với doanh nghiệp vừa và nhỏ, quy trình đơn giản nhưng đầy đủ.
- Ứng dụng AI (ML và LLM) giúp nâng cao giá trị và tính khác biệt.
- Giao diện thân thiện, phân quyền rõ ràng, dễ vận hành.
- Nền tảng web giúp truy cập linh hoạt trên nhiều thiết bị.

Hạn chế:

- Chưa bao phủ toàn bộ các module của ERP.
- Module dự báo cần dữ liệu thực tế để đạt độ chính xác cao hơn.
- Chưa kiểm thử tích hợp phần cứng RFID.
- Khả năng chịu tải lớn cần được đánh giá thêm.

8.3 Bài học kinh nghiệm

Thông qua quá trình thực hiện, nhóm đã hiểu rõ hơn về các đánh đổi trong kiến trúc service-based, đặc biệt là vấn đề transaction phân tán và tính nhất quán dữ liệu. Việc sử dụng Kafka giúp nhóm tiếp cận và làm chủ mô hình event-driven trong hệ thống thực tế.

Bên cạnh đó, nhóm cũng tích lũy kinh nghiệm trong việc ứng dụng LLM, từ thiết kế prompt đến đánh giá kết quả. Đồng thời, việc kết hợp phát triển và kiểm thử song song giúp nâng cao chất lượng sản phẩm và giảm thiểu lỗi.

8.4 Hướng phát triển

Trong tương lai, hệ thống có thể được mở rộng theo các hướng sau:

- Xây dựng module quản lý mua hàng và nhà cung cấp.
- Nâng cấp module dự báo với dữ liệu thực tế và các yếu tố như mùa vụ.
- Tích hợp chức năng bán hàng và thương mại điện tử.
- Bổ sung quản lý sản xuất và nguyên vật liệu.
- Hoàn thiện thành hệ thống ERP toàn diện.

8.5 Kết luận

Đồ án đã xây dựng thành công một hệ thống WMS hoạt động ổn định trên nền kiến trúc hiện đại. Điểm nổi bật không chỉ nằm ở việc triển khai hệ thống quản lý kho, mà còn ở việc tích hợp các kỹ thuật AI vào quá trình phân tích và hỗ trợ vận hành.

Stockify hướng tới trở thành một giải pháp phù hợp cho doanh nghiệp vừa và nhỏ tại Việt Nam, đồng thời giúp nhóm tích lũy kinh nghiệm thực tiễn về phát triển hệ thống, lựa chọn công nghệ và tư duy thiết kế phần mềm trong môi trường thực tế.